

# AdaptiveMaxPool2d#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.pooling.AdaptiveMaxPool2d.html>

## Description:

Applies a 2D adaptive max pooling over an input signal composed of several input planes. The output is of size  $H_{out} \times W_{out}$ , for any input size. The number of output features is equal to the number of input planes. `output_size` (Union[int, None, tuple[Optional[int], Optional[int]]]) – the target output size of the image of the form  $H_{out} \times W_{out}$ . Can be a tuple  $(H_{out}, W_{out})$  or a single  $H_{out}$  for a square image  $H_{out} \times H_{out}$ .  $H_{out}$  and  $W_{out}$  can be either a int, or None which means the size will be the same as that of the input. `return_indices` (bool) – if True, will return the indices along with the outputs. Useful to pass to `nn.MaxUnpool2d`. Default: False Input:  $(N, C, H_{in}, W_{in})$  or  $(N, C, H_{in}, W_{in})$  or  $(C, H_{in}, W_{in})$ .

## Function Signature

---

```
class  
torch.nn.modules.pooling.AdaptiveMaxPool2d(output_size,  
return_indices=False)[source]#
```

Parameters

Shape:

```
forward(input)[source]#
```

## Code Examples

---

```
>>> # target output size of 5x7  
>>> m = nn.AdaptiveMaxPool2d((5, 7))  
>>> input = torch.randn(1, 64, 8, 9)  
>>> output = m(input)  
>>> # target output size of 7x7 (square)  
>>> m = nn.AdaptiveMaxPool2d(7)  
>>> input = torch.randn(1, 64, 10, 9)  
>>> output = m(input)  
>>> # target output size of 10x7  
>>> m = nn.AdaptiveMaxPool2d((None, 7))  
>>> input = torch.randn(1, 64, 10, 9)  
>>> output = m(input)
```

# Detailed Documentation

---

## Documentation

Applies a 2D adaptive max pooling over an input signal composed of several input planes.

The output is of size  $H_{out} \times W_{out}$ , for any input size. The number of output features is equal to the number of input planes.

`output_size` (Union[int, None, tuple[Optional[int], Optional[int]]]) – the target output size of the image of the form  $H_{out} \times W_{out}$ . Can be a tuple  $(H_{out}, W_{out})$  or a single  $H_{out}$  for a square image  $H_{out} \times H_{out}$ .  $H_{out}$  and  $W_{out}$  can be either a int, or None which means the size will be the same as that of the input.

`return_indices` (bool) – if True, will return the indices along with the outputs. Useful to pass to `nn.MaxUnpool2d`. Default: False

Input:  $(N, C, H_{in}, W_{in})$  or  $(C, H_{in}, W_{in})$

Output:  $(N, C, H_{out}, W_{out})$  or  $(C, H_{out}, W_{out})$ , where  $(H_{out}, W_{out}) = \text{output\_size}(H_{out}, W_{out})$ .

Runs the forward pass.

---

